

Set Operations in PostgreSQL: UNION, INTERSECT, EXCEPT

Table of Contents

1. [Learning Objectives](#)
 2. [Theory: Set Theory in SQL](#)
 3. [UNION](#)
 4. [UNION ALL](#)
 5. [INTERSECT](#)
 6. [INTERSECT ALL](#)
 7. [EXCEPT](#)
 8. [EXCEPT ALL](#)
 9. [Combining Multiple Set Operations](#)
 10. [ASCII Visual Diagrams](#)
 11. [Set Operations vs JOINS](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Hands-on Exercises](#)
 17. [Advanced Notes](#)
 18. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Use UNION, UNION ALL, INTERSECT, and EXCEPT correctly
 - Understand deduplication behavior (ALL vs without ALL)
 - Choose between set operations and JOINS for given problems
 - Order and paginate set operation results
 - Apply set operations to real reporting and data quality scenarios
-

Theory: Set Theory in SQL

SQL set operations combine results of two or more SELECT statements vertically (stacking rows). They are rooted in mathematical set theory.

Rules for All Set Operations

1. Both queries must return the same NUMBER of columns.
2. Corresponding columns must have COMPATIBLE data types.
3. Column NAMES come from the FIRST query (left side).
4. ORDER BY applies to the FINAL combined result only (place at the very end, not inside each query).

5. UNION/INTERSECT/EXCEPT deduplicate rows.
UNION ALL/INTERSECT ALL/EXCEPT ALL preserve duplicates.

Sample Tables

```
-- Employees from 2023 and 2024 (simulating historical data)
CREATE TEMP TABLE emp_2023 (
    employee_id INT,
    employee_name VARCHAR(50),
    department VARCHAR(30),
    salary NUMERIC(10,2)
);

CREATE TEMP TABLE emp_2024 (
    employee_id INT,
    employee_name VARCHAR(50),
    department VARCHAR(30),
    salary NUMERIC(10,2)
);

INSERT INTO emp_2023 VALUES
    (1, 'Alice', 'Engineering', 90000),
    (2, 'Bob', 'Engineering', 85000),
    (3, 'Carol', 'Marketing', 70000),
    (4, 'David', 'Sales', 65000),
    (5, 'Eve', 'HR', 60000);

INSERT INTO emp_2024 VALUES
    (2, 'Bob', 'Engineering', 88000), -- salary raise
    (3, 'Carol', 'Marketing', 70000), -- unchanged
    (4, 'David', 'Engineering', 72000), -- transferred
    (6, 'Frank', 'Sales', 67000), -- new hire
    (7, 'Grace', 'Marketing', 75000), -- new hire

-- City lists for set operation demos
CREATE TEMP TABLE cities_a (city VARCHAR(30));
CREATE TEMP TABLE cities_b (city VARCHAR(30));

INSERT INTO cities_a VALUES ('New York'), ('Chicago'), ('Los Angeles'), ('Houston');
INSERT INTO cities_b VALUES ('Chicago'), ('Houston'), ('Phoenix'), ('Dallas');
```

UNION

UNION combines results of two queries and **removes duplicate rows**.

ASCII Diagram

Query A results:	Query B results:
New York	Chicago

Chicago		Houston	
Los Angeles		Phoenix	
Houston		Dallas	

UNION result (duplicates removed):

New York	
Chicago	← appears once (deduped from both)
Los Angeles	
Houston	← appears once (deduped from both)
Phoenix	
Dallas	

Examples

```
-- Example 1: Basic UNION – all cities, deduplicated
SELECT city FROM cities_a
UNION
SELECT city FROM cities_b
ORDER BY city;

-- Example 2: UNION to combine employee lists from two years
SELECT employee_id, employee_name, department, '2023' AS year
FROM emp_2023
UNION
SELECT employee_id, employee_name, department, '2024' AS year
FROM emp_2024
ORDER BY employee_name, year;

-- Example 3: UNION with WHERE filters
SELECT employee_name, department FROM emp_2023 WHERE department = 'Engineering'
UNION
SELECT employee_name, department FROM emp_2024 WHERE department = 'Engineering'
ORDER BY employee_name;

-- Example 4: UNION combining heterogeneous sources with CAST
SELECT product_id::TEXT AS entity_id, product_name AS entity_name, 'product' AS type
FROM products
UNION
SELECT customer_id::TEXT, customer_name, 'customer'
FROM customers
ORDER BY type, entity_name;

-- Example 5: UNION for a report combining current and archived data
-- (assumes an archived_orders table exists)
SELECT order_id, customer_id, order_date, total_amount, 'current' AS source
FROM orders
UNION
SELECT order_id, customer_id, order_date, total_amount, 'archived'
```

```
FROM orders    -- substitute archived_orders in real scenario
WHERE order_date < '2023-01-01'
ORDER BY order_date DESC;
```

UNION ALL

UNION ALL stacks all rows from both queries **without deduplication**. Faster than UNION.

Examples

```
-- Example 6: UNION ALL – preserve all rows including duplicates
SELECT city FROM cities_a
UNION ALL
SELECT city FROM cities_b
ORDER BY city;
-- Chicago and Houston appear TWICE

-- Example 7: UNION ALL for event log combining multiple sources
SELECT 'login' AS event_type, user_id, event_time FROM login_events
UNION ALL
SELECT 'purchase', user_id, event_time FROM purchase_events
UNION ALL
SELECT 'logout', user_id, event_time FROM logout_events
ORDER BY event_time;

-- Example 8: UNION ALL in a CTE for cross-type reporting
WITH all_transactions AS (
    SELECT order_id AS txn_id, total_amount, order_date FROM orders
    UNION ALL
    SELECT sale_id, amount, sale_date FROM sales
)
SELECT
    DATE_TRUNC('month', txn_date) AS month,
    COUNT(*) AS transactions,
    SUM(amount) AS total
FROM (
    SELECT txn_id, total_amount AS amount, order_date AS txn_date FROM orders
    UNION ALL
    SELECT sale_id, amount, sale_date FROM sales
) AS combined
GROUP BY DATE_TRUNC('month', txn_date)
ORDER BY month;

-- Example 9: Performance test – UNION vs UNION ALL
EXPLAIN ANALYZE
SELECT city FROM cities_a UNION SELECT city FROM cities_b;
-- Note the Sort + Unique (or HashAggregate) step for deduplication

EXPLAIN ANALYZE
```

```
SELECT city FROM cities_a UNION ALL SELECT city FROM cities_b;
-- Note NO dedup step – much cheaper
```

INTERSECT

INTERSECT returns only rows that appear in **both** query results (set intersection).

ASCII Diagram

Query A:

New York
Chicago
Los Angeles
Houston



Query B:

Chicago
Houston
Phoenix
Dallas



INTERSECT result:

Chicago
Houston

← in both A and B

← in both A and B

Examples

```
-- Example 10: Cities in both lists
SELECT city FROM cities_a
INTERSECT
SELECT city FROM cities_b;

-- Example 11: Employees who appear in both 2023 and 2024 (by ID – survivors)
SELECT employee_id FROM emp_2023
INTERSECT
SELECT employee_id FROM emp_2024;

-- Example 12: Employees with identical records in both years (no change)
SELECT employee_id, employee_name, department, salary FROM emp_2023
INTERSECT
SELECT employee_id, employee_name, department, salary FROM emp_2024;
-- Returns Carol (unchanged across both years)

-- Example 13: Products ordered by BOTH customer 1 and customer 2
SELECT product_id FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
WHERE o.customer_id = 1
INTERSECT
SELECT product_id FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
WHERE o.customer_id = 2;
```

INTERSECT ALL

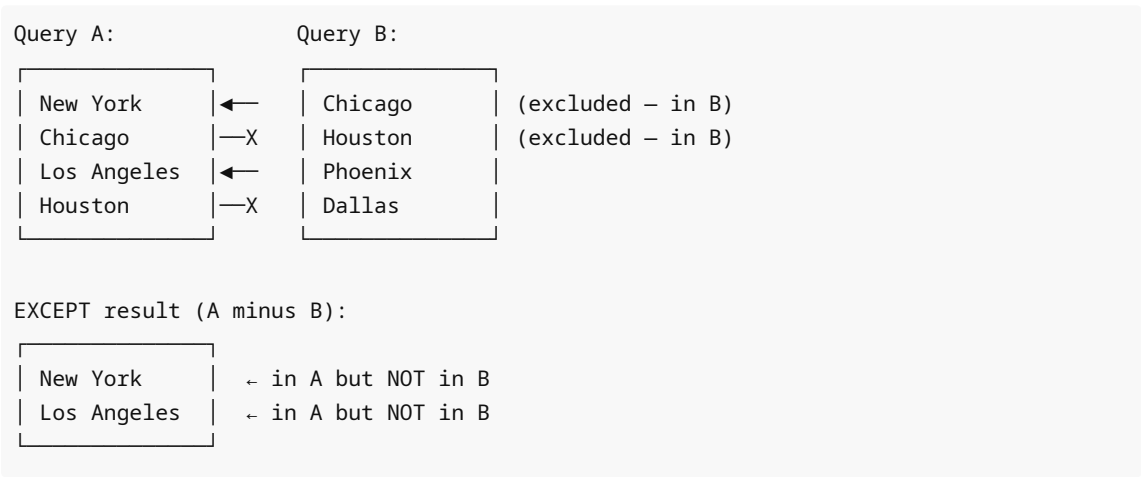
INTERSECT ALL returns the intersection while **preserving duplicate counts** (minimum count from each side).

```
-- Example 14: INTERSECT ALL – preserves duplicate multiplicity
WITH data_a AS (VALUES ('X'), ('X'), ('Y'), ('Z')),
     data_b AS (VALUES ('X'), ('Y'), ('Y'))
SELECT * FROM data_a
INTERSECT ALL
SELECT * FROM data_b;
-- X appears min(2,1) = 1 time
-- Y appears min(1,2) = 1 time
-- Z not in data_b → excluded
```

EXCEPT

EXCEPT returns rows that appear in the **first query but not in the second** (set difference).

ASCII Diagram



Examples

```
-- Example 15: Cities only in list A (not in B)
SELECT city FROM cities_a
EXCEPT
SELECT city FROM cities_b;

-- Example 16: Employees who left between 2023 and 2024
SELECT employee_id, employee_name FROM emp_2023
EXCEPT
SELECT employee_id, employee_name FROM emp_2024;
-- Returns Alice, Eve (in 2023 but not in 2024)

-- Example 17: Customers who have NOT ordered (anti-join via EXCEPT)
```

```

SELECT customer_id FROM customers
EXCEPT
SELECT customer_id FROM orders WHERE customer_id IS NOT NULL;

-- Example 18: Data quality – find rows in staging not in production
-- (assumes staging_employees table)
SELECT employee_id, employee_name, salary FROM emp_2023 -- staging
EXCEPT
SELECT employee_id, employee_name, salary FROM emp_2024 -- production
ORDER BY employee_id;

-- Example 19: EXCEPT for change detection (compare snapshots)
SELECT * FROM emp_2023
EXCEPT
SELECT * FROM emp_2024;
-- Returns all rows that changed or were deleted between the two snapshots

```

EXCEPT ALL

EXCEPT ALL subtracts occurrences while preserving remaining duplicates.

```

-- Example 20: EXCEPT ALL – subtract one occurrence at a time
WITH data_a AS (VALUES ('X'), ('X'), ('X'), ('Y')),
     data_b AS (VALUES ('X'), ('X'))
SELECT * FROM data_a
EXCEPT ALL
SELECT * FROM data_b;
-- X: 3 in A minus 2 in B = 1 remaining X
-- Y: 1 in A, 0 in B = 1 Y
-- Result: X, Y

```

Combining Multiple Set Operations

```

-- Example 21: Chain of set operations
-- Cities in A or B, but not in both (symmetric difference)
(SELECT city FROM cities_a EXCEPT SELECT city FROM cities_b)
UNION
(SELECT city FROM cities_b EXCEPT SELECT city FROM cities_a)
ORDER BY city;

-- Example 22: UNION with INTERSECT – precedence
-- INTERSECT has HIGHER precedence than UNION (like multiplication vs addition)
SELECT city FROM cities_a
UNION
SELECT city FROM cities_b
INTERSECT
SELECT city FROM (VALUES ('Chicago'), ('Dallas')) AS c(city);
-- Parsed as: cities_a UNION (cities_b INTERSECT {Chicago, Dallas})

```

```
-- Use parentheses to control order explicitly:
(SELECT city FROM cities_a UNION SELECT city FROM cities_b)
INTERSECT
SELECT city FROM (VALUES ('Chicago'), ('Dallas')) AS c(city);
```

Set Operations vs JOINS

```
-- EXCEPT equivalent using anti-join (NOT EXISTS):
SELECT employee_id FROM emp_2023
EXCEPT
SELECT employee_id FROM emp_2024;
-- Equivalent JOIN-based anti-join:
SELECT e23.employee_id FROM emp_2023 e23
WHERE NOT EXISTS (SELECT 1 FROM emp_2024 e24 WHERE e24.employee_id =
e23.employee_id);

-- INTERSECT equivalent using semi-join (EXISTS):
SELECT employee_id FROM emp_2023
INTERSECT
SELECT employee_id FROM emp_2024;
-- Equivalent:
SELECT DISTINCT e23.employee_id FROM emp_2023 e23
WHERE EXISTS (SELECT 1 FROM emp_2024 e24 WHERE e24.employee_id = e23.employee_id);

-- When to use set operations vs JOINS:
-- ✓ Set ops: combining same-shaped results from different sources/tables
-- ✓ Set ops: dedup between two queries naturally
-- ✓ JOINS: when you need columns from BOTH tables in the output
-- ✓ JOINS: for complex multi-column matching conditions
```

Common Mistakes

Mistake 1: Column count mismatch

```
-- WRONG: different number of columns
SELECT id, name FROM table_a
UNION
SELECT id FROM table_b; -- ERROR: different number of columns

-- CORRECT: pad with NULL or a literal
SELECT id, name FROM table_a
UNION
SELECT id, NULL::VARCHAR AS name FROM table_b;
```

Mistake 2: ORDER BY inside a set operation member


```
-- WRONG: ORDER BY inside a UNION member
SELECT name FROM emp_2023 ORDER BY name
UNION
SELECT name FROM emp_2024;

-- CORRECT: ORDER BY at the end applies to the whole result
SELECT name FROM emp_2023
UNION
SELECT name FROM emp_2024
ORDER BY name;
```

Mistake 3: Using UNION when UNION ALL is intended for performance

```
-- UNION adds a costly dedup step (Sort + Unique or Hash)
-- If you know there are no duplicates OR want all rows, use UNION ALL
SELECT * FROM orders_q1
UNION ALL -- no dedup needed for disjoint date ranges
SELECT * FROM orders_q2;
```

Mistake 4: Type incompatibility

```
-- WRONG: incompatible types (INT vs VARCHAR)
SELECT employee_id FROM emp_2023
UNION
SELECT employee_name FROM emp_2024; -- type mismatch

-- CORRECT: explicit CAST
SELECT employee_id::TEXT FROM emp_2023
UNION
SELECT employee_name FROM emp_2024;
```

Mistake 5: INTERSECT precedence

```
-- INTERSECT binds more tightly than UNION – always use parentheses!
-- Ambiguous:
SELECT a FROM t1 UNION SELECT a FROM t2 INTERSECT SELECT a FROM t3;

-- Explicit:
SELECT a FROM t1 UNION (SELECT a FROM t2 INTERSECT SELECT a FROM t3);
```

Best Practices

1. Use UNION ALL instead of UNION when duplicates are not a concern — it is significantly faster.
2. Always use parentheses with mixed UNION/INTERSECT/EXCEPT for clarity.
3. Put ORDER BY only at the end of the entire set operation query.
4. Use explicit CAST when column types differ between queries.
5. Leverage EXCEPT for change detection between table snapshots.

6. When INTERSECT is semantically the same as EXISTS, compare performance with EXPLAIN.
 7. Name columns descriptively in the first SELECT — they set the column names for all.
-

Performance Considerations

```
-- UNION requires deduplication (extra Sort or Hash step)
-- UNION ALL is always faster when duplicates are acceptable

-- Example: Check the plan difference
EXPLAIN (ANALYZE, COSTS)
SELECT employee_name FROM emp_2023 UNION SELECT employee_name FROM emp_2024;
-- Plan shows: HashAggregate for dedup

EXPLAIN (ANALYZE, COSTS)
SELECT employee_name FROM emp_2023 UNION ALL SELECT employee_name FROM emp_2024;
-- Plan shows: Append node only (no dedup)

-- INTERSECT and EXCEPT also deduplicate – consider EXISTS/NOT EXISTS for correlated
scenarios

-- Indexes do NOT directly speed up set operation dedup
-- but do speed up underlying table scans if WHERE conditions are pushed down

-- For large result set unions, consider:
-- 1. Partition tables and use UNION ALL across partitions
-- 2. Use UNION ALL then GROUP BY to manually dedup on desired keys
SELECT employee_name, MAX(salary) AS latest_salary
FROM (
    SELECT employee_name, salary FROM emp_2023
    UNION ALL
    SELECT employee_name, salary FROM emp_2024
) AS combined
GROUP BY employee_name;
```

Interview Questions & Answers

Q1: What is the difference between UNION and UNION ALL?

A: UNION deduplicates the combined result set (using a Sort + Unique or HashAggregate), returning each unique row once. UNION ALL preserves all rows from both queries, including duplicates, and requires no dedup step — making it significantly faster. Use UNION when you need deduplication; use UNION ALL when you know results are disjoint or duplicates are acceptable.

Q2: What are the rules for using set operations?

A: Both queries must return the same number of columns, and corresponding columns must have compatible data types. Column names are determined by the first query. ORDER BY can only appear once, at the very end.

Q3: What is the operator precedence for UNION, INTERSECT, EXCEPT?

A: INTERSECT has higher precedence than UNION and EXCEPT. UNION and EXCEPT have equal precedence and are evaluated left to right. Always use parentheses to make the intended order explicit.

Q4: How would you implement a symmetric difference (items in A or B but not both)?

A:

```
(SELECT * FROM A EXCEPT SELECT * FROM B)
UNION
(SELECT * FROM B EXCEPT SELECT * FROM A);
```

Q5: How is EXCEPT different from NOT IN?

A: EXCEPT is a set operation that compares entire rows and deduplicates. NOT IN compares a single column and has the NULL trap (returns no rows if the subquery contains NULLs). EXCEPT is safer for multi-column row comparison. NOT IN is simpler for single-column comparisons when NULLs are controlled.

Q6: When would you use INTERSECT instead of an INNER JOIN?

A: INTERSECT is useful when you want identical rows that exist in two result sets (same shape). INNER JOIN is used when you want columns from both tables based on a key relationship. INTERSECT is often used for data comparison between two snapshots.

Q7: Can you use ORDER BY inside a UNION member query?

A: No. ORDER BY can only appear at the end of the complete set operation. Each individual member query cannot have its own ORDER BY (unless wrapped in a subquery, though this has no guaranteed effect without LIMIT).

Q8: What does EXCEPT ALL do differently from EXCEPT?

A: EXCEPT ALL subtracts rows one-for-one based on occurrence count, not just membership. If row X appears 3 times in A and 1 time in B, EXCEPT ALL returns X twice. EXCEPT returns X zero times (deduplicates).

Hands-on Exercises

Exercise 1: Employee Roster Changes

Find employees who were in 2023 but NOT in 2024 (departed), and employees who are in 2024 but NOT in 2023 (new hires). Label each.

```
-- Solution:
SELECT employee_name, 'Departed' AS status FROM emp_2023
EXCEPT
SELECT employee_name, 'Departed' FROM emp_2024
UNION ALL
SELECT employee_name, 'New Hire' FROM emp_2024
EXCEPT ALL
SELECT employee_name, 'New Hire' FROM emp_2023
ORDER BY status, employee_name;

-- Cleaner approach:
(SELECT employee_name, 'Departed' AS status FROM emp_2023
```

```

    EXCEPT SELECT employee_name, 'Departed' FROM emp_2024)
UNION ALL
(SELECT employee_name, 'New Hire' AS status FROM emp_2024
 EXCEPT SELECT employee_name, 'New Hire' FROM emp_2023)
ORDER BY status, employee_name;

```

Exercise 2: Common Departments

Find department names that exist in both the 2023 and 2024 employee data.

```

-- Solution:
SELECT DISTINCT department FROM emp_2023
INTERSECT
SELECT DISTINCT department FROM emp_2024
ORDER BY department;

```

Exercise 3: Unique City List

Combine cities_a and cities_b into one deduplicated list, sorted alphabetically.

```

-- Solution:
SELECT city FROM cities_a
UNION
SELECT city FROM cities_b
ORDER BY city;

```

Exercise 4: Data Quality Check

Check whether the set of customer IDs in the customers table exactly matches the set of customer IDs in the orders table (finding mismatches in both directions).

```

-- Solution:
SELECT customer_id, 'Missing in orders' AS issue
FROM customers
WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders WHERE customer_id
IS NOT NULL)

UNION ALL

SELECT customer_id, 'Missing in customers' AS issue
FROM (
    SELECT DISTINCT customer_id FROM orders WHERE customer_id IS NOT NULL
    EXCEPT
    SELECT customer_id FROM customers
) AS orphan
ORDER BY issue, customer_id;

```

Exercise 5: UNION ALL Performance Test

Write a query that combines orders from two hypothetical quarter tables (simulate with WHERE filters on the orders table) using UNION ALL, then aggregate the combined result.

```
-- Solution:
SELECT
    DATE_TRUNC('month', order_date) AS month,
    COUNT(*) AS orders,
    SUM(total_amount) AS revenue
FROM (
    SELECT order_id, order_date, total_amount
    FROM orders
    WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31'
    UNION ALL
    SELECT order_id, order_date, total_amount
    FROM orders
    WHERE order_date BETWEEN '2024-04-01' AND '2024-06-30'
) AS h1_orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

Advanced Notes

CORRESPONDING BY (Standard SQL)

Standard SQL supports `UNION CORRESPONDING BY (column_list)` to match columns by name rather than position. PostgreSQL does not support this syntax — use explicit column lists instead.

Set Operations with NULL

NULL values are treated as equal in set operations (unlike normal equality comparisons):

```
-- These NULLs are deduplicated by UNION:
SELECT NULL::INT UNION SELECT NULL::INT;
-- Returns: 1 row (one NULL)

-- In normal WHERE comparison: NULL = NULL → UNKNOWN
-- In set dedup: NULLs are considered identical
```

Materialized CTEs and Set Operations

```
WITH MATERIALIZED snapshot_2023 AS (SELECT * FROM emp_2023),
     MATERIALIZED snapshot_2024 AS (SELECT * FROM emp_2024)
SELECT * FROM snapshot_2023
EXCEPT
SELECT * FROM snapshot_2024;
```

Cross-references

- [04_exists_any_all.md](#) — EXISTS/NOT EXISTS as alternatives to INTERSECT/EXCEPT

- **01_joins_complete.md** — JOINS as alternatives for INTERSECT/EXCEPT patterns
- **03_subqueries.md** — Subquery equivalents of set operations
- **04_ctes.md** (04_Advanced_SQL) — CTEs combined with set operations